PARCIAL PRÁCTICO

Detección de Fraude Bancario

Machine Learning con PySpark y Docker

Universidad Santo Tomás | Estadística | 2026-I

OBJETIVO

Construir un modelo de Machine Learning para detectar transacciones bancarias fraudulentas en un dataset altamente desbalanceado (~0.17% de fraudes), comparando 3 modelos diferentes y entregando las predicciones para evaluación tipo competencia Kaggle.

El problema

Una entidad bancaria europea registra millones de transacciones diarias. Detectar manualmente las fraudulentas es imposible. La tarea es construir un sistema automatizado que identifique transacciones sospechosas con la mayor precisión posible, minimizando los falsos negativos (fraudes no detectados) sin generar demasiadas falsas alarmas.

Los datos

Archivo	Filas	Descripción
train.csv	**199,365**	Datos de entrenamiento CON labels (Class)
test.csv	**42,720**	Datos de prueba SIN labels (debes predecir)
sample_submission.csv	**42,720**	Ejemplo del formato de entrega

Variables (features)

- Time: segundos transcurridos desde la primera transacción
- V1, V2, ..., V28: features numéricas resultado de PCA (anonimizadas por privacidad)
- Amount: monto de la transacción
- Class: target binario — 0 = transacción normal, 1 = fraude (solo en train.csv)

REGLAS DEL PARCIAL

1. Modelos requeridos (mínimo 3)

#	Modelo	Framework	Obligatoriedad
1	**Logistic Regression (baseline)**	PySpark	**Obligatorio**
2	**Random Forest o XGBoost (GBT)**	PySpark	**Obligatorio**
3	**Su modelo asignado**	PySpark o scikit-learn	**Obligatorio**

2. Asignación individual de modelos

Estudiante	Modelo	Framework
Kevin Chaparro	**LightGBM**	scikit-learn
Sergio Prieto	**CatBoost**	scikit-learn
Jhonatan Cañón	**MLP (Red Neuronal)**	PySpark
Josué Pedraza	**Stacking**	Manual
Valentina Muñoz	**LinearSVC**	PySpark
Camilo Velandia	**AdaBoost**	scikit-learn
Sergio Huertas	**Ridge (L2)**	PySpark
Mildreth Díaz	**Lasso (L1)**	PySpark
Natalia Zárate	**Elastic Net**	PySpark
Michael Morantes	**Naive Bayes**	PySpark
Daniela Murcia	**KNN**	scikit-learn

PySpark vs scikit-learn

Aspecto	PySpark MLlib	scikit-learn
Procesamiento	Distribuido (clúster)	Memoria local (1 máquina)
Tamaño de datos	TB / millones de filas	MB / miles a cientos de miles
Variedad de modelos	Limitada (~15 algoritmos)	Muy amplia (>30 algoritmos)
Rapidez de iteración	Más lenta (overhead de Spark)	Muy rápida
Cuándo usarla	Producción, big data, ETL	Prototipado, modelos especializados

Estrategia recomendada para este parcial

1. EDA y preparación de datos: SIEMPRE en PySpark (es nuestra base).
2. Modelos 1 y 2 (Logistic + RF/XGBoost): en PySpark (existen ahí).
3. Modelo 3 (tu asignado):

- Si está en PySpark → úsalo en PySpark (mantienes coherencia)
- Si NO está en PySpark → convierte a Pandas con .toPandas() y usa scikit-learn

ENTREGABLES

Estructura del repositorio

Crea un repositorio en GitHub con la siguiente estructura:

```
TuNombre_ParcialPractico/
├── notebooks/
│   └── parcial_TuNombre.ipynb
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
├── submission.csv         ← predicciones del MEJOR modelo
├── reporte.pdf            ← 1 página máximo
└── README.md
```

Formato del submission.csv

DEBE tener exactamente estas columnas y 42,720 filas (más el header):

id	Class
1	0
2	0
3	**1**
...	...
42720	0

Reporte (1 página máximo)

- Resumen del problema y datos
- Tabla comparativa de los 3 modelos (F1, Precision, Recall, AUC)
- Justificación estadística de tu mejor modelo
- Recomendación al banco (¿qué modelo poner en producción y por qué?)

CALIFICACIÓN

Distribución de la nota (sobre 5.0)

Criterio	Peso	¿Cómo se evalúa?
F1-Score en leaderboard	**40%**	Cálculo automático sobre test
Calidad del notebook (EDA + código)	**30%**	Limpieza, justificación, rigor
Comparación de los 3 modelos	**20%**	Tabla con métricas + análisis
Docker funcional	**10%**	Levanta sin errores y corre el notebook

¿Cómo se asigna la nota del leaderboard?

Se asigna en ORDEN según el F1-Score obtenido sobre el conjunto de prueba oculto:

• El estudiante con mejor F1 obtiene la nota máxima de esta sección.

• Los demás reciben una nota proporcional a su posición y a su F1 relativo al máximo.

• Una entrega con formato incorrecto o sin submission obtiene 1 en esta sección.

Es decir: el orden del leaderboard determina el orden de las notas en el componente F1.

PREGUNTAS DE SUSTENTACIÓN

Pregunta 1 — Bitácora de experimentos

Incluya en su notebook una sección titulada "Bitácora de Experimentos" con la siguiente tabla obligatoria. Mínimo 5 filas, una por experimento realizado al ajustar SU modelo asignado:

#	Hiperparámetro modificado	Valor anterior	Valor nuevo	F1 (validation)	Decisión
1					
2					
3					
4					
5					

Los valores de F1 reportados deben coincidir EXACTAMENTE con los outputs ejecutados en el notebook. Cualquier inconsistencia entre lo escrito en la bitácora y los outputs del notebook se considera entrega no válida.

Pregunta 2 — Semilla personal

En todas las divisiones aleatorias (randomSplit, train_test_split, etc.) de su notebook, debe usar como seed los últimos 4 dígitos de su número de cédula. Reportar:

a) ¿Cuál es su semilla?

b) Con esa semilla, ¿cuántas filas tiene su train interno y su validation?

c) ¿Cuántos fraudes (Class=1) quedaron en cada conjunto?

d) Con esa semilla, ¿cuál fue el F1 EXACTO (4 decimales) de su modelo asignado en validation?

Pregunta 3 — Análisis de errores específicos

Después de evaluar su mejor modelo en validation, identifique:

a) Los **3 IDs específicos** del conjunto de validation donde su modelo predijo Class=1 (fraude) con probabilidad mayor a 0.9 pero la realidad era Class=0 (Falsos Positivos con alta confianza).

b) Los **3 IDs específicos** donde su modelo predijo Class=0 (normal) pero la realidad era Class=1 (Falsos Negativos — fraudes que se le escaparon al modelo).

c) Para uno de esos 6 casos, observa los valores de las features (V1-V28, Amount, Time) y plantee una hipótesis estadística sobre por qué el modelo pudo equivocarse.

Esta pregunta requiere ejecutar el modelo con SU semilla — los IDs serán distintos para cada estudiante.